Project 1. CSR SpMV: Memory Footprint and Cache Mapping Analysis

1. Reproduction Flow

```
$ bash run.sh
```

`run.sh` builds and verifies the benchmark through `scripts/verify_benchmark.py`: it compiles with strict warnings, runs the independent CSR self-test, and checks that local and permuted layouts produce the same checksum. It then runs identical workloads for the local and permuted layouts (`65536` rows, `8` nonzeros per row, `500` iterations), logging the verifier and benchmark output under `logs/`. Add `--pmu` when hardware counters are useful and available; PMU availability is reported as a status and is not required for correctness.

2. Workload

The workload is a compressed sparse row (CSR) sparse matrix-vector multiplication. In SpMV_Guided_Benchmark.cpp, `row_ptr` identifies the start and end of each matrix row, while `column_index` and `values` store the nonzero entries. The function `csr_spmv` accumulates one output value in `sum` for each `row`:

$$\begin{aligned}
&\text{for } row = 0, \ldots, \texttt{row_ptr.size()} - 2, \\
&\quad \texttt{sum} = \sum_{k=\texttt{row_ptr}[row]}^{\texttt{row_ptr}[row+1]-1} \texttt{values}[k] \times \texttt{x}[\texttt{column_index}[k]], \\
&\quad \texttt{y}[row] = \texttt{sum}.
\end{aligned}$$

The source-code loop therefore performs the equivalent update `sum += values[k] * x[column_index[k]]` for every stored nonzero. In the illustrated row 3 example, the nonzeros are `6` and `7` in columns `2` and `4`, so the computation is

$$\texttt{y[3]} = 6 \times \texttt{x[2]} + 7 \times \texttt{x[4]} = 6 \times 3 + 7 \times 5 = 53, \qquad \texttt{x} = [1, 2, 3, 4, 5].$$

During setup, `rows` determines the number of outputs in `y`, `column_count` determines the length of `x`, and `nnz_per_row` determines the fixed row width. Thus `nonzero_count = rows * nnz_per_row`, and `expected` stores an independent logical-order result used to validate the kernel before and after the timed loop.

3. Memory Model

The benchmark uses a CSR matrix with `R = 65,536` rows, `C = 4R = 262,144` columns, and exactly eight nonzeros per row. Therefore, `NNZ = R * 8 = 524,288`. The executable stores `row_ptr` as 64-bit offsets, `col_idx` as 32-bit indices, and `values`, `x`, and `y` as 64-bit values. The `expected` array is an additional 64-bit correctness oracle and is not kernel payload. Sizes below use binary units (`1 KiB = 1,024 bytes`); the logical access calculation counts every indexed `x` read and must not be labeled as DRAM traffic.

Array	Count	Element size	Computation	Bytes
`row_ptr`	`R + 1 = 65,537`	8 B	`65,537 * 8`	`524,296` B (`512 KiB + 8 B`)
`col_idx`	`NNZ = 524,288`	4 B	`524,288 * 4`	`2,097,152` B (`2 MiB`)
`values`	`NNZ = 524,288`	8 B	`524,288 * 8`	`4,194,304` B (`4 MiB`)
`x`	`C = 262,144`	8 B	`262,144 * 8`	`2,097,152` B (`2 MiB`)
`y`	`R = 65,536`	8 B	`65,536 * 8`	`524,288` B (`512 KiB`)
`expected`	`R = 65,536`	8 B	`65,536 * 8`	`524,288` B (`512 KiB`)
Total	**six arrays**	—	**sum of all arrays**	**9,961,480 B (≈9.50 MiB)**

One complete SpMV call performs `NNZ = 524,288` logical reads from `x`, so its logical `x` demand is `524,288 * 8 = 4,194,304` bytes (`4 MiB`). This is a logical demand count, not a claim about the amount of data transferred from DRAM.

4. A Cache Mapping Toy Example

Logical request size is not the same as DRAM traffic: caches at each level can retain data and enable reuse, allowing later requests to be served without reaching DRAM. The required toy trace, `[0, 1, 8, 9, 0, 8, 1, 9]` repeated three times, studies this behavior using a simplified single-level cache access pattern.

Assume element addresses start at `0x1000`. **The assignment's cache trace uses 4-byte elements, a 16-byte cache line, and a 32-byte direct-mapped cache; these toy-cache assumptions are independent of the real benchmark's 8-byte doubles.** Thus there are two cache lines and two resident slots. For element index `i`, `address` is the element's byte address, `block_id` identifies its 16-byte memory block relative to the base address, `offset` is the element's byte position within that cache line, and `set` identifies the direct-mapped cache slot to which the block is assigned:

$$\begin{aligned} \text{address} &= 0x1000 + 4i, \\ \text{block_id} &= \left\lfloor \frac{\text{address} - 0x1000}{16} \right\rfloor, \\ \text{offset} &= (\text{address} - 0x1000) \bmod 16, \\ \text{set} &= \text{block_id} \bmod 2. \end{aligned}$$

The absolute memory block IDs are 256 and 258 for relative block IDs 0 and 2; both map to resident direct-mapped slot 0. `H` and `M` denote hit and miss. The fully associative comparison has the same two-line capacity and uses LRU replacement.

#	Element	Address	Block ID	Offset	Set	Direct mapped	Fully associative
1	0	`0x1000`	0	0	0	M	M
2	1	`0x1004`	0	4	0	H	H
3	8	`0x1020`	2	0	0	M	M
4	9	`0x1024`	2	4	0	H	H

#	Element	Address	Block ID	Offset	Set	Direct mapped	Fully associative
5	0	0x1000	0	0	0	**M**	**H**
6	8	0x1020	2	0	0	**M**	**H**
7	1	0x1004	0	4	0	**M**	**H**
8	9	0x1024	2	4	0	**M**	**H**
9	0	0x1000	0	0	0	**M**	**H**
10	1	0x1004	0	4	0	H	H
11	8	0x1020	2	0	0	**M**	**H**
12	9	0x1024	2	4	0	H	H
13	0	0x1000	0	0	0	**M**	**H**
14	8	0x1020	2	0	0	**M**	**H**
15	1	0x1004	0	4	0	**M**	**H**
16	9	0x1024	2	4	0	**M**	**H**
17	0	0x1000	0	0	0	**M**	**H**
18	1	0x1004	0	4	0	H	H
19	8	0x1020	2	0	0	**M**	**H**
20	9	0x1024	2	4	0	H	H
21	0	0x1000	0	0	0	**M**	**H**
22	8	0x1020	2	0	0	**M**	**H**
23	1	0x1004	0	4	0	**M**	**H**
24	9	0x1024	2	4	0	**M**	**H**

The direct-mapped cache has 6 hits and 18 misses: blocks 0 and 2 continually evict one another from slot 0. The equal-capacity fully associative cache has 22 hits and 2 compulsory misses because both blocks remain resident after their first accesses. This illustrates why associativity matters even when total cache capacity is unchanged.